

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y COMPUTACIÓN
SISTEMAS EMBEBIDOS

TAREA 2

Comunicación entre microcontroladores

Integrantes:	Carolina Sánchez Jhony Choez Elián Montes María del Carmen Garzón
Nombre del proyecto:	Juego de la serpiente.
Docente:	MSc. Ronald Solis.
Paralelo:	1

1. Objetivos

1.1 Objetivo general

Desarrollar un sistema de juego interactivo del clásico juego de la serpiente (Snake) con tres niveles de dificultad, empleando el microcontrolador ATmega328P para el control visual mediante una matriz LED 8x8 y el microcontrolador PIC16F887 para la reproducción de melodías asociadas a los eventos del juego, estableciendo una comunicación entre ambos dispositivos con el fin de reforzar habilidades en programación de sistemas embebidos, diseño de circuitos y simulación en Proteus.

1.2 Objetivos específicos

- Implementar el control de una matriz LED 8x8 con el ATmega328P para visualizar el movimiento de la serpiente, la comida, los niveles de dificultad y los estados del juego.
- Programar el PIC16F887 para reproducir melodías diferenciadas según los eventos del juego: comer fruta y colisión o derrota.
- Establecer un canal de comunicación unidireccional entre el ATmega328P y el PIC16F887 mediante la línea PC4, utilizando pulsos de duración variable para sincronizar los efectos visuales y sonoros en tiempo real.
- Integrar cuatro pulsadores conectados al ATmega328P para controlar la dirección de la serpiente (arriba, derecha, abajo, izquierda) mediante interrupciones de cambio de pin.
- Diseñar e implementar tres niveles de dificultad que modifiquen dinámicamente la velocidad de desplazamiento de la serpiente, aumentando el desafío progresivamente entre el nivel fácil, medio y difícil.

2. Descripción técnica del juego

El proyecto implementa el clásico juego *Snake* (Serpiente) sobre una matriz LED 8x8. El objetivo es guiar a la serpiente para que consuma la mayor cantidad de frutas posible sin chocar contra las paredes ni contra su propio cuerpo. La cantidad de frutas requeridas para ganar varía según el nivel de dificultad seleccionado. Al cumplir la condición de victoria se muestra una carita feliz; al morir, una carita triste.

2.1. Arquitectura del sistema

El diseño se basa en una arquitectura de doble microcontrolador que separa las responsabilidades de lógica/visual y audio:

- **Controlador Principal (ATmega328P):** Es el cerebro del sistema. Gestiona la totalidad de la lógica del juego:
 - Animación de título: Al encender el sistema despliega el texto "SNAKE" desplazándose de derecha a izquierda en la matriz LED.
 - Menú de dificultad: Muestra el signo "?" y espera que el jugador presione uno de los tres primeros botones para seleccionar el nivel (FAC / MED / DIF). Confirma la selección con el cuarto botón tras mostrar el signo "!".

- **Motor gráfico:** Realiza el multiplexado de la matriz LED dibujando la serpiente y la comida mediante barrido de filas.
 - **Lógica de juego:** Procesa las entradas de los botones (arriba, derecha, abajo, izquierda) mediante interrupciones (PCINT), actualiza la posición de la serpiente, detecta colisiones con paredes y cuerpo propio, y gestiona la generación aleatoria de comida.
 - **Señalización de audio:** Envía pulsos de duración específica al PIC a través del pin PC4 para indicar los eventos comer (10 ms), perder (90 ms) y ganar (1000 ms).
-
- **Coprocador de Audio (PIC16F887):** Actúa como procesador de sonido dedicado. Mide la duración del pulso recibido en RC0 y reproduce el tono correspondiente a través del altavoz conectado a RC2:
 - Pulso corto (50–250 ms): tono de comer (frecuencia alta, breve).
 - Pulso medio (250–750 ms): secuencia descendente de tonos de derrota.
 - Pulso largo (≥ 750 ms): fanfarria de victoria (4 notas ascendentes).

2.1 Funcionamiento y Jugabilidad

Al encender el sistema, el ATmega328P despliega la animación del nombre del juego "SNAKE" una sola vez, desplazándose horizontalmente en la matriz. A continuación, aparece el signo "?", indicando al jugador que puede seleccionar su nivel de dificultad:

- Botón 1 (Arriba/PC0): Selecciona nivel Fácil → muestra "FAC" y luego "!".
- Botón 2 (Derecha/PC1): Selecciona nivel Medio → muestra "MED" y luego "!".
- Botón 3 (Abajo/PC2): Selecciona nivel Difícil → muestra "DIF" y luego "!".
- Botón 4 (Izquierda/PC3): Confirma la selección cuando el signo "!" está visible e inicia el juego.

Una vez iniciado el juego, la serpiente comienza con longitud 3 en posición central, moviéndose hacia la derecha. El jugador la dirige con los cuatro botones. Las reglas son:

- La serpiente muere si choca contra cualquier pared (borde de la matriz) o contra su propio cuerpo.
- Al comer una fruta, la serpiente crece en una unidad y se genera una nueva fruta en posición aleatoria libre.
- Al alcanzar la longitud objetivo según el nivel, el jugador gana.

2.2. Niveles de Dificultad

Fácil: Velocidad de 200 unidades de refresco. La serpiente debe alcanzar longitud 5 (comer 2 frutas).

Medio: Velocidad de 120 unidades de refresco. La serpiente debe alcanzar longitud 6 (comer 3 frutas).

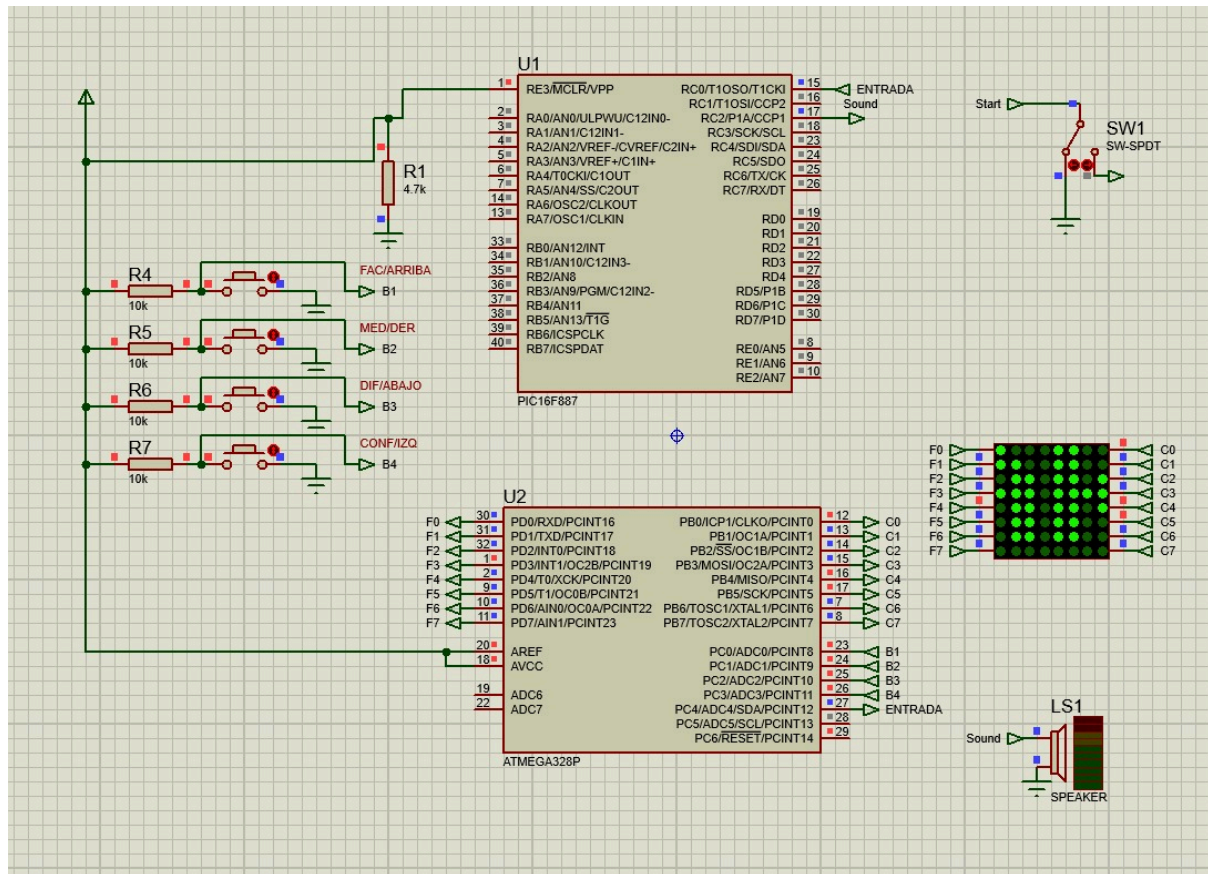


Figura 2. Animación del título "SNAKE" desplazándose en la matriz LED.

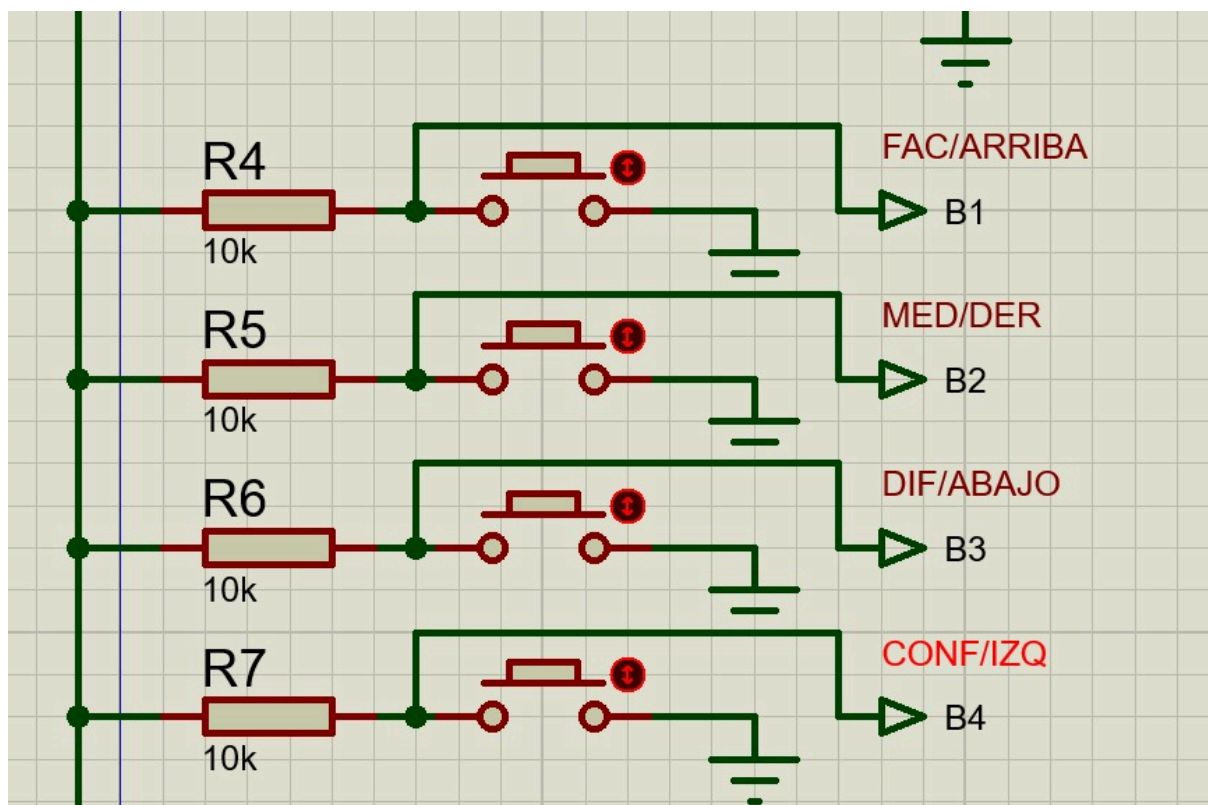


Figura 3. Menú de selección de nivel y partida en curso.

4. Explicación del código

4.1 ATmega 328P

Bloque 1: Configuración de Hardware y Variables Globales

- Pines de entrada (botones): PC0–PC3 se configuran como entradas con resistencias pull-up internas activadas. El botón PC4 (SOUND_PIN) se configura como salida para la señal de audio hacia el PIC.
- Interrupciones: Se habilitan interrupciones por cambio de pin (PCINT) para PC0–PC3, permitiendo detectar pulsaciones de botón de forma inmediata sin bloquear el bucle principal.
- Semilla aleatoria: Se inicializa el registro LFSR con la lectura del ADC en canal flotante (ADC7), garantizando posiciones de comida distintas en cada partida.
- Variables del juego: gameboard almacena la posición de cada segmento de la serpiente (valor > 0 indica cuerpo, el valor más alto es la cabeza). snakeLength, direction, snakeSpeed y longitudParaGanar controlan la lógica del juego.

Bloque 2: Recursos Visuales (Letras, Caritas y Motor Gráfico)

- Sprites en PROGMEM: Los patrones de bits para las letras S, N, A, K, E, F, A, C, M, E, D, I, los signos ? y !, y las caritas feliz/triste se almacenan en la memoria de programa (PROGMEM) para no consumir RAM.
- Función mostrarImagen(): Realiza el multiplexado de la matriz. Activa cada fila en PORTD y escribe el byte invertido en PORTB. Se repite 30 veces para reducir el parpadeo visible.
- Función mostrarSnakeDesplazando(): Desplaza horizontalmente las letras "SNAKE" de izquierda a derecha a través de la matriz. Calcula en tiempo real qué columna de qué letra debe mostrarse en cada posición de la matriz para el desplazamiento actual.
- Función refrescarPantalla(): Es el motor gráfico del juego. Recorre el arreglo gameboard e ilumina los LEDs correspondientes a la serpiente; adicionalmente enciende el LED de la comida (foodRow, foodCol). El número de iteraciones depende de snakeSpeed, creando así la diferencia de velocidad entre niveles.

Bloque 3: Lógica del Juego

- Función inicializarJuego(): Según el nivel seleccionado asigna snakeSpeed (200/120/60) y longitudParaGanar (5/6/8). Reinicia el tablero, posiciona la serpiente inicial de longitud 3 en el centro y coloca la primera fruta en (5,5).
- Función actualizarSerpiente(): Calcula la nueva posición de la cabeza según la dirección actual. Verifica si la nueva posición es un borde (muerte por pared) o si ya contiene un segmento de la serpiente (muerte por auto-colisión). Si se come la fruta, incrementa snakeLength y llama a generarComida(). El movimiento se implementa decrementando todos los valores del tablero (el cero indica celda vacía) y escribiendo snakeLength en la nueva posición de la cabeza.
- ISR(PCINT0_vect): Captura la pulsación de botones por interrupción. Almacena la dirección solicitada en pendingDirection, validando que no sea el reverso directo de la dirección actual (evita que la serpiente se voltee sobre sí misma).
- Función generarComida(): Genera coordenadas aleatorias para la nueva fruta, asegurándose de que la celda esté vacía. Si snakeLength ≥ longitudParaGanar, activa la bandera win y termina el juego.

Bloque 4: Menú de Dificultad

- `función menuDificultad()`: Muestra el signo "?" e implementa un bucle de lectura por flanco de botón (detección de transición 0→1). Al presionar uno de los tres primeros botones muestra la abreviatura del nivel ("FAC", "MED" o "DIF") y el signo "!". El cuarto botón (PC3) confirma la selección y retorna al bucle principal únicamente si ya se eligió un nivel.

Bloque 5: Señalización de Audio hacia el PIC

- `sonidoComer()`: Pone PC4 en HIGH durante 10 ms y regresa a LOW. El PIC mide un pulso de 10 ms y ejecuta el tono de comer.
- `sonidoPerder()`: Pulso de 90 ms. El PIC ejecuta la secuencia descendente de tonos de derrota.
- `sonidoGanar()`: Pulso de 1000 ms. El PIC ejecuta la fanfarria de victoria de 4 notas.

4.2 PIC16F887

Bloque 1: Configuración de Pines

- `TRISC0 = 1` (entrada): RC0 recibe la señal de duración desde el ATmega328P.
- `TRISC2 = 0` (salida): RC2 está conectado al altavoz. La función `Sound_Init()` de MikroC inicializa este pin para la generación de tonos.
- `ANSEL / ANSELH = 0`: Deshabilita todas las entradas analógicas para que los pines operen en modo digital.

Bloque 2: Medición del Pulso – `medirPulso()`

Esta función es el núcleo del protocolo de comunicación. Mientras la señal en RC0 permanece en HIGH, incrementa un contador con resolución de 1 ms. Retorna el tiempo medido (en milisegundos) y tiene un límite máximo de 1500 ms para evitar bloqueos indefinidos en caso de falla de comunicación.

Bloque 3: Reproducción de Sonidos

- `playEatSound()`: Reproduce una nota de 2500 Hz durante 120 ms. Tono agudo y breve que confirma visualmente que se comió una fruta.
- `playLoseSound()`: Reproduce una secuencia descendente de cuatro notas (900, 700, 500, 250 Hz) con pausas de 30 ms entre ellas. Transmite sensación de derrota.
- `playWinSound()`: Reproduce cuatro notas ascendentes (523, 659, 784, 1047 Hz) correspondientes a Do-Mi-Sol-Do de la escala mayor. La última nota tiene mayor duración (300 ms) para dar énfasis al logro.

Bloque 4: Bucle Principal y Decodificación del Protocolo

El `main()` del PIC opera en un bucle continuo de espera activa:

1. Espera un flanco de subida en RC0 (`SIGNAL_IN == 1`).
2. Llama a `medirPulso()` para obtener la duración del pulso en ms.
3. Aplica los umbrales de decisión:
 - 50–249 ms → comer (`playEatSound`)
 - 250–749 ms → perder (`playLoseSound`)

- ≥ 750 ms $\rightarrow$ ganar (playWinSound)
4. Espera a que RC0 vuelva a LOW para evitar detección doble.
 5. Aplica un tiempo muerto de 100 ms antes de volver a escuchar.

5. Esquema de comunicación entre microcontroladores

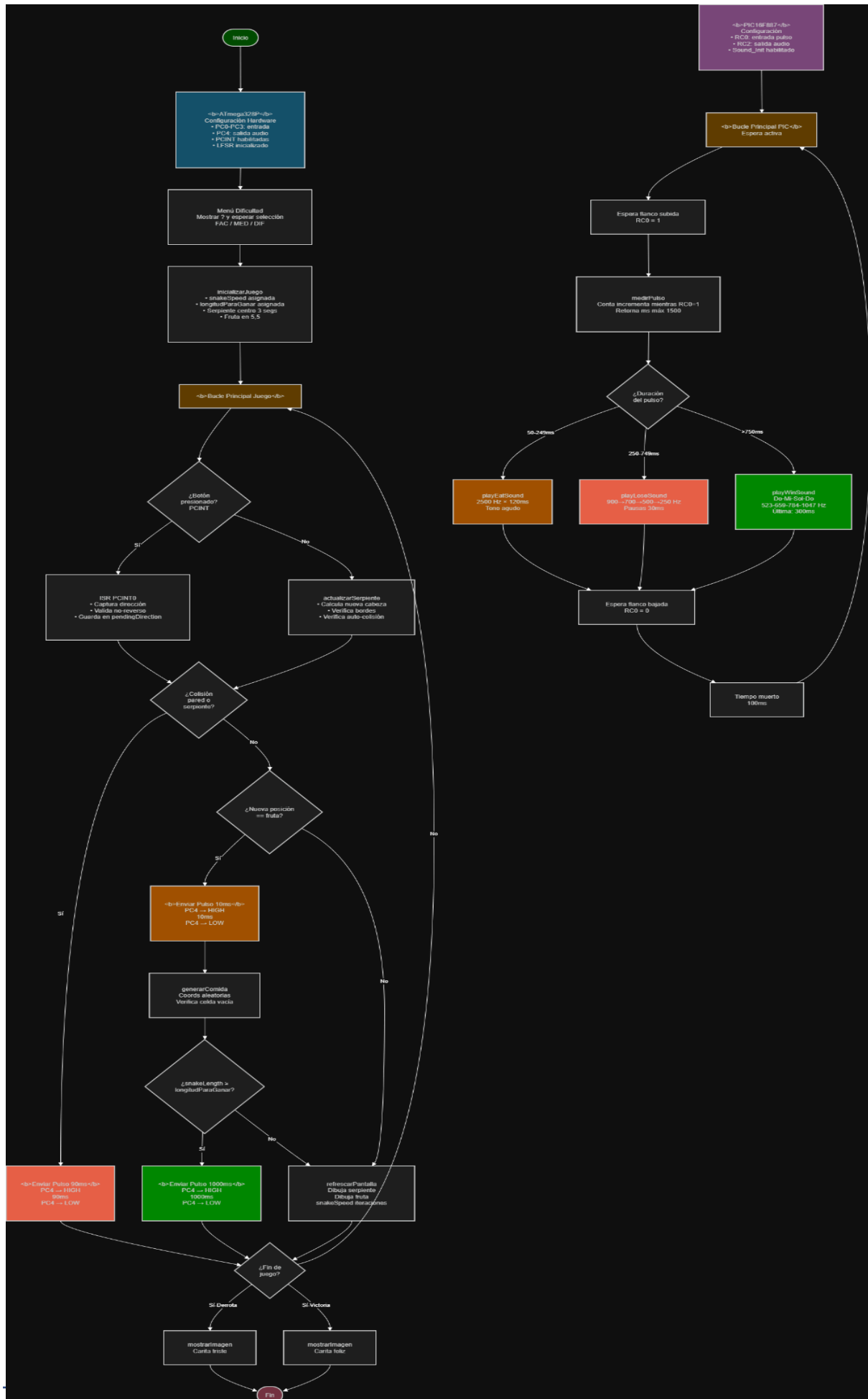


Figura 4. Diagrama de flujo del protocolo de comunicación

6. Repositorio de GitHub

El código fuente completo, archivos de simulación Proteus y evidencias adicionales están disponibles en: <https://github.com/carisanc/Sistemas-Embebidos---Tarea-2/tree/main>

El repositorio contiene:

- Código fuente del ATmega328P.
- Código fuente del PIC16F887.
- Archivo de simulación Proteus (.pdsprj)
- El video para visualizar el funcionamiento del juego

7. Conclusiones y recomendaciones

7.1 Conclusiones

1. Se implementó exitosamente el juego Snake en una matriz LED 8×8, con animación de título, menú de selección de nivel y lógica completa de juego (movimiento, colisiones, comida y condición de victoria), todo gestionado por el ATmega328P.
2. El ATmega328P demostró ser suficientemente capaz para manejar simultáneamente el multiplexado de la matriz LED, la detección de entradas por interrupción, la lógica de movimiento de la serpiente y la generación de señales de control, sin degradar la experiencia de juego.
3. El PIC16F887 funcionó efectivamente como coprocesador de audio dedicado. La delegación de la generación de sonido a este dispositivo permitió al ATmega dedicar todos sus ciclos al motor gráfico y la lógica del juego, evitando pausas o parpadeos causados por las funciones de sonido bloqueantes.
4. El protocolo de comunicación por duración de pulso demostró ser una solución simple, robusta y económica en pines para sincronizar dos microcontroladores con funciones distintas. Con un único cable se transmiten tres eventos diferenciados de manera confiable.
5. Los tres niveles de dificultad lograron una diferenciación perceptible en la experiencia de juego, tanto por la velocidad de la serpiente como por el número de frutas requeridas, cumpliendo el objetivo pedagógico de la tarea.

7.2 Recomendaciones

1. Para una versión futura, se recomienda reemplazar el protocolo de pulso por una comunicación UART o I2C, lo que permitiría transmitir comandos más complejos y reducir la dependencia de temporización precisa.
2. Se sugiere implementar la generación de tonos del PIC mediante interrupciones de Timer en lugar de usar el Sound_Play, creando un sistema de audio no bloqueante que permita al PIC realizar otras tareas en paralelo, como gestionar un LED de estado.
3. El código del ATmega podría extenderse para incluir un marcador de puntuación (score) representado en la matriz, o para añadir obstáculos internos en niveles superiores, incrementando la complejidad del juego.

4. Para mejorar la robustez del hardware, se sugiere agregar condensadores de desacoplo en los pines de alimentación de ambos microcontroladores y resistencias de protección en los pines de la matriz LED.